

RFC-001: SoftMeter — Plataforma de Metrologia de Software

Engenharia de Software – Católica SC

Identificação

- **Título do Projeto:**
SoftMeter — Plataforma de Inspeção e Metrologia de Qualidade de Software
 - **Linha de Projeto (Direction):**
Web App
 - **Autor:**
Diego Cunha
 - **Data da Proposta:**
12/04/2026
 - **Versão:**
1.0
-

1. Visão do Produto e Impacto (O Problema)

O objetivo desta seção é responder uma pergunta fundamental:

Este projeto resolve um problema real ou é apenas um exercício técnico?

É um problema real, vivido diariamente por equipes de desenvolvimento e gestores de TI: softwares são entregues como "prontos" sem que existam critérios objetivos e quantitativos que comprovem essa afirmação.

1.1 Contexto e Problema

A indústria manufatureira opera há décadas sob um princípio inegociável: nenhuma peça é aceita sem medição. Tolerâncias dimensionais, laudos de inspeção e certificações metrológicas garantem que cada componente entregue atende precisamente ao que foi projetado. Um furo fora de centro por décimos de milímetro é motivo de rejeição imediata — não por subjetividade, mas porque existe um protocolo claro de medição e conformidade.

A Engenharia de Software, por sua vez, enfrenta um paradoxo persistente: **sistemas são entregues como "prontos" e "funcionando" sem que critérios objetivos de aceitação tenham sido rigorosamente definidos e medidos**. O que significa exatamente que um software é "rápido o suficiente"? Quantos erros de

entrada de dados são toleráveis? Em que momento uma falha intermitente passa de "comportamento esperado" para "defeito crítico"?

Na ausência de métricas quantitativas claras, a qualidade de software torna-se uma percepção — e percepções variam entre clientes, desenvolvedores e gestores.

Quem sofre com esse problema:

- Gestores de TI que não conseguem provar objetivamente se o sistema entregue atende ao contrato
- Equipes de QA que realizam testes sem critérios numéricos de aceitação definidos
- Clientes que recebem software "funcionando" mas com desempenho aquém do esperado
- Desenvolvedores que não sabem quando parar de otimizar por falta de uma "tolerância" clara

Como é resolvido atualmente:

- Testes informais baseados em percepção subjetiva do usuário
- Planilhas de acompanhamento sem cálculo automático de conformidade
- SLAs genéricos definidos em contrato mas nunca medidos sistematicamente
- Ferramentas de monitoramento que coletam dados mas não emitem laudos de conformidade

Limitações das soluções atuais:

- Ausência de um protocolo estruturado que ligue requisito → medição → desvio → laudo
- Ferramentas de QA focam em encontrar bugs, não em medir conformidade quantitativa
- Nenhuma ferramenta disponível traduz os conceitos maduros da metrologia industrial para o contexto de software

Exemplo real do problema:

Um sistema de ERP entregue a uma empresa de médio porte apresenta tempo de carregamento de tela de 4,2 segundos. O contrato estipulava "sistema ágil e responsivo". Não existe definição numérica, não existe tolerância, não existe laudo — e o cliente reclamará de lentidão enquanto o fornecedor argumentará que o sistema "está funcionando". Com o SoftMeter, o requisito seria: **Tempo de carregamento: Nominal 2,00s | Tolerância máxima 2,50s | Medido 4,20s | Desvio +2,20s | Status: REPROVADO.**

1.2 Origem da Demanda e Evidências

É necessário demonstrar que existe **interesse real pela solução**.

Demanda Externa — Contexto Profissional do Autor

O projeto nasce da experiência profissional direta do autor como **técnico de metrologia dimensional** — operador de Braço de Medição FARO e software PolyWorks em ambiente de fabricação de peças de precisão (Quality Ferramentaria, Joinville/SC) — que, durante a transição de carreira para a Engenharia de Software, identificou a ausência de rigor metrológico nas práticas de QA das empresas que conheceu.

A observação central motivadora: **a mesma empresa que rejeitaria uma peça por 0,1 mm de desvio aceita um software com 40% dos requisitos não-funcionais fora do especificado**, simplesmente porque não existe um protocolo de inspeção equivalente.

Pesquisa com Usuários

A proposta foi apresentada a profissionais de tecnologia com perfis alinhados às personas do projeto, incluindo analistas de QA, desenvolvedores e gestores de TI. As principais dores identificadas foram:

Dor relatada	Frequência
"Não sei como justificar para o cliente que o sistema está dentro do esperado"	Alta
"Fazemos testes mas não temos critérios numéricos de aceitação"	Alta
"O SLA está no contrato mas nunca medimos se estamos cumprindo"	Média
"Não sei quando uma funcionalidade está 'boa o suficiente' para subir para produção"	Média

Padrão observado: todos os participantes reconheceram o conceito de tolerância metrológica aplicada ao software como intuitivo e de fácil compreensão, especialmente após a analogia com a metrologia industrial.

Evidência de Interesse Acadêmico

A crítica de Sommerville (2011) — *"a ideia de tolerâncias não é aplicável aos sistemas digitais"* — evidencia exatamente o gap que este projeto se propõe a preencher: não há ferramentas que tornem essa aplicação prática e acessível. O SoftMeter posiciona-se como uma resposta direta a essa lacuna identificada na literatura.

1.3 Análise de Soluções Existentes (Benchmark)

SonarQube / SonarCloud

- **Link:** <https://www.sonarqube.org>
- **Público-alvo:** Equipes de desenvolvimento
- **Funcionalidades:** Análise estática de código, detecção de bugs, code smells, cobertura de testes, métricas de dívida técnica
- **Limitações:** Foca exclusivamente em qualidade de código-fonte. Não mede conformidade de requisitos não-funcionais em produção. Não emite laudos de conformidade por requisito.

New Relic / Datadog

- **Link:** <https://newrelic.com> / <https://datadoghq.com>
- **Público-alvo:** Times de DevOps e SRE
- **Funcionalidades:** Monitoramento de performance em tempo real, alertas, dashboards de observabilidade, rastreamento de erros
- **Limitações:** Focado em observabilidade operacional contínua. Não permite definir tolerâncias por requisito nem gerar laudos comparativos formais entre versões. Não produz documentação de conformidade para entrega ao cliente.

Jira + Xray (Test Management)

- **Link:** <https://www.atlassian.com/software/jira>

- **Público-alvo:** Equipes de QA e desenvolvimento ágil
- **Funcionalidades:** Gestão de casos de teste, execução, rastreabilidade de requisitos, relatórios de cobertura
- **Limitações:** Resultado binário (passou/falhou). Não quantifica desvios, não calcula índice de conformidade percentual, não emite laudo metrológico formal.

TestRail

- **Link:** <https://www.testrail.com>
- **Público-alvo:** Gestores de QA
- **Funcionalidades:** Planejamento e execução de testes, relatórios detalhados, integração com ferramentas de desenvolvimento
- **Limitações:** Mesma limitação do Jira/Xray — resultado binário, sem conceito de tolerância ou desvio metrológico.

Microsoft Azure Test Plans

- **Link:** <https://azure.microsoft.com/en-us/products/devops/test-plans>
- **Público-alvo:** Times que utilizam o ecossistema Microsoft/Azure DevOps
- **Funcionalidades:** Testes manuais e automatizados, rastreabilidade, integração com pipelines CI/CD
- **Limitações:** Vinculado ao ecossistema Azure. Não possui conceito de valor nominal, tolerância ou laudo de conformidade metrológico.

Comparação

Solução	Pontos Fortes	Limitações
SonarQube	Análise de código profunda, integração CI/CD	Sem métricas de requisitos não-funcionais em produção
New Relic / Datadog	Dados de performance em tempo real	Sem tolerâncias definíveis por requisito, sem laudo formal
Jira + Xray	Gestão de testes integrada ao desenvolvimento	Resultado binário, sem cálculo de desvio
TestRail	Relatórios de QA completos	Sem conformidade quantitativa metrológica
Azure Test Plans	Integração nativa com DevOps	Vinculado ao ecossistema Microsoft, sem conceito metrológico

Diferencial do Projeto

O **SoftMeter** preenche uma lacuna específica não atendida por nenhuma das soluções acima:

1. **Define tolerâncias por requisito** — cada requisito tem um valor nominal, tolerância superior e inferior, assim como uma cota dimensional em metrologia industrial
2. **Calcula desvios automaticamente** — a diferença entre o valor medido e o esperado é calculada e classificada (Aprovado / Alerta / Reprovado)

3. **Emita Laudos de Conformidade** — documento estruturado com índice percentual de conformidade e classificação final (Conforme $\geq 90\%$ / Condicional 70–89% / Não-Conforme $< 70\%$)
 4. **Rastreabilidade metrológica completa** — cada medição é vinculada a um ciclo de teste documentado com versão do software e ambiente de teste
 5. **Acessível a pequenas equipes** — sem necessidade de infraestrutura complexa ou licenças caras
-

1.4 Público-Alvo

Perfil primário — Analistas de QA / Testers

- Profissionais responsáveis pela garantia de qualidade de sistemas
- Contexto: equipes de desenvolvimento de software em empresas de pequeno e médio porte
- Nível técnico: intermediário — conhecem testes de software, não necessariamente metrologia
- Principal ganho: critérios objetivos e documentação formal de conformidade

Perfil secundário — Gestores de TI / Product Owners

- Precisam provar para o cliente que o software entregue atende ao contratado
- Contexto: entrega de projetos para clientes externos, especialmente órgãos públicos
- Nível técnico: básico-intermediário — precisam de relatórios claros, não de dados técnicos brutos
- Principal ganho: Laudo de Conformidade formal para uso em entregas contratuais

Perfil terciário — Desenvolvedores de software

- Querem saber exatamente quando uma funcionalidade está "dentro da tolerância" antes do deploy
 - Contexto: times ágeis com foco em qualidade contínua
 - Nível técnico: avançado
 - Principal ganho: critério objetivo de "pronto" para cada funcionalidade
-

1.5 Objetivos do Projeto

Objetivo Geral

Desenvolver o **SoftMeter**, uma plataforma web que implementa o framework de Metrologia de Software: um protocolo estruturado que transpõe os conceitos da metrologia industrial (tolerância, desvio, conformidade, laudo) para a Engenharia de Software, permitindo avaliar quantitativamente se um sistema atende aos seus requisitos especificados e gerar Laudos de Conformidade de Software objetivos e rastreáveis.

Objetivos Específicos

- Mapear e transpor os conceitos do Vocabulário Internacional de Metrologia (VIM) para equivalentes no domínio de software, fundamentando o framework teórico com base na norma ISO/IEC 25010
- Implementar o cadastro de sistemas e requisitos com definição de valores nominais e tolerâncias (superiores e inferiores) por requisito, com cálculo automático de desvio e classificação de conformidade
- Implementar o registro de ciclos de teste e medições com geração automática de Laudos de Conformidade de Software em formato exportável (PDF)

- Validar o framework por meio de um estudo de caso aplicado a um sistema real, demonstrando a geração de um laudo completo com índice de conformidade calculado
- Disponibilizar a plataforma em ambiente de produção acessível via URL pública, com pipeline CI/CD configurado e cobertura de testes unitários $\geq 75\%$ no backend

1.6 Métricas de Sucesso (KPIs)

KPI	Meta
Tempo de resposta da API	< 300ms para 95% das requisições
Disponibilidade do sistema	$\geq 99\%$ em ambiente de produção
Cobertura de testes backend (TDD)	$\geq 75\%$
Tempo para gerar um laudo completo	< 5 segundos
Redução do tempo de documentação de QA	$\geq 50\%$ vs processo manual em planilha
Número de requisitos suportados por ciclo	Sem limite definido (escalável)

2. Engenharia de Requisitos

Esta seção define **o que o sistema fará**.

2.1 Personas

Persona 1 — Ana Souza, Analista de QA

- **Idade:** 28 anos
 - **Contexto:** Trabalha em uma softwarehouse de médio porte, responsável pelos testes de 3 produtos simultaneamente. Não tem equipe dedicada de QA — acumula a função com outras responsabilidades.
 - **Objetivos:** Ter critérios claros de aceitação para cada funcionalidade e documentar resultados de forma profissional para apresentar ao cliente.
 - **Principais dificuldades:** Testes baseados em "achismo", sem métricas definidas. Relatórios finais são subjetivos e contestados pelo cliente após a entrega.
-

Persona 2 — Carlos Mendes, Gerente de TI

- **Idade:** 42 anos
 - **Contexto:** Gerencia a entrega de sistemas para prefeituras e órgãos públicos, que exigem laudos técnicos formais como parte do processo de aceite contratual.
 - **Objetivos:** Ter documentação que prove que o software entregue atende ao contrato e reduzir as reclamações pós-entrega.
 - **Principais dificuldades:** Não existe nenhum "documento de conformidade" padronizado que os desenvolvedores emitam. Reclamações pós-entrega são frequentes e custosas.
-

Persona 3 — Rafael Lima, Desenvolvedor Backend

- **Idade:** 25 anos, em início de carreira
- **Contexto:** Desenvolve APIs para um sistema de gestão e quer garantir qualidade antes do deploy em produção.
- **Objetivos:** Saber se sua API está "dentro da tolerância" de desempenho antes de subir para produção.
- **Principais dificuldades:** Apenas testa se "funciona" — não tem base para dizer se o desempenho está "bom o suficiente" e quando pode considerar a tarefa concluída.

2.2 Casos de Uso Principais

1. Criar conta e autenticar-se na plataforma
2. Cadastrar sistema de software para avaliação
3. Definir requisitos com tolerâncias (cotas de software)
4. Registrar ciclo de teste com identificação de ambiente e versão
5. Registrar medições por requisito dentro do ciclo
6. Visualizar desvios calculados automaticamente
7. Encerrar ciclo e gerar Laudo de Conformidade
8. Exportar laudo em PDF
9. Comparar laudos entre versões do mesmo sistema

Fluxo principal:

```
Login → Dashboard → Selecionar/Criar Sistema → Definir Requisitos  
→ Abrir Ciclo de Teste → Registrar Medições → Encerrar Ciclo  
→ Visualizar Laudo → Exportar PDF
```

2.3 Requisitos Funcionais (RF)

Gestão de Sistemas

RF01 — O sistema deve permitir que o usuário cadastre sistemas de software com nome, descrição e responsável.

RF02 — O sistema deve permitir que o usuário liste, edite e archive sistemas cadastrados.

Gestão de Requisitos (Cotas de Software)

RF03 — O sistema deve permitir que o usuário cadastre requisitos vinculados a um sistema, com: código, descrição, valor nominal, tolerância superior, tolerância inferior, unidade de medida e criticidade.

RF04 — O sistema deve permitir que o usuário classifique requisitos como Crítico, Maior ou Menor.

RF05 — O sistema deve permitir que o usuário edite e desative requisitos sem perder o histórico de medições.

Ciclos de Teste

RF06 — O sistema deve permitir que o usuário registre ciclos de teste vinculados a um sistema, com versão do software e descrição do ambiente de teste.

RF07 — O sistema deve permitir que o usuário encerre um ciclo de teste, bloqueando novas medições após o encerramento.

Medições

RF08 — O sistema deve permitir que o usuário registre medições por requisito dentro de um ciclo, informando o valor medido e observações opcionais.

RF09 — O sistema deve calcular automaticamente o desvio absoluto (medido – nominal), o desvio relativo (%) e o status de conformidade (Aprovado / Alerta / Reprovado) para cada medição.

Laudos

RF10 — O sistema deve gerar automaticamente um Laudo de Conformidade ao encerrar um ciclo, com índice percentual de conformidade e classificação final.

RF11 — O sistema deve permitir que o usuário visualize e exporte o laudo em PDF.

RF12 — O sistema deve permitir que o usuário compare laudos de dois ciclos diferentes do mesmo sistema.

Autenticação

RF13 — O sistema deve exigir autenticação para acesso a qualquer funcionalidade.

RF14 — O sistema deve permitir que o usuário crie conta com e-mail e senha.

2.4 Requisitos Não Funcionais (RNF)

RNF01 — O sistema deve responder a 95% das requisições de API em menos de 300ms.

RNF02 — O sistema deve estar disponível em ambiente de produção via URL pública (cloud).

RNF03 — O sistema deve utilizar autenticação com JWT e HTTPS obrigatório em produção.

RNF04 — O sistema deve ter cobertura de testes unitários de no mínimo 75% no backend.

RNF05 — O sistema deve ter pipeline CI/CD configurado via GitHub Actions com execução automática de testes a cada push.

RNF06 — O sistema deve utilizar banco de dados PostgreSQL em produção.

RNF07 — O sistema deve ser responsivo e utilizável em desktop e tablet.

RNF08 — O sistema deve aplicar análise estática de código (SonarCloud ou equivalente).

RNF09 — O sistema deve ter monitoramento básico de performance e disponibilidade em produção.

2.5 Regras de Negócio

RN01 — Um ciclo de teste encerrado não pode receber novas medições.

RN02 — Um laudo só pode ser gerado se o ciclo tiver ao menos uma medição para cada requisito ativo.

RN03 — O status "Alerta" é atribuído quando o valor medido está a menos de 5% do limite de tolerância.

RN04 — O Índice de Conformidade Geral = (requisitos Aprovados / total de requisitos avaliados) × 100.

RN05 — Classificação final: Conforme (≥ 90%), Condicional (70%–89%), Não-Conforme (< 70%).

RN06 — Qualquer requisito com criticidade "Crítico" reprovado resulta em classificação Não-Conforme, independente do índice geral.

RN07 — Apenas o usuário proprietário do sistema pode cadastrar, editar ou encerrar ciclos desse sistema.

2.6 Fora do Escopo

- Integração automática com ferramentas de CI/CD para coleta passiva de medições (os valores são inseridos manualmente pelo usuário nesta versão)
- Execução automatizada de testes de carga ou stress
- Suporte a múltiplos idiomas (apenas português brasileiro)
- Aplicativo mobile nativo (iOS/Android)
- Gestão financeira ou contratual
- Integração com ferramentas de ticketing (Jira, Azure DevOps)
- Relatórios de cobertura de código (escopo do SonarQube)

3. Fluxos e Comportamento do Sistema

Esta seção demonstra **como o sistema funciona**.

3.1 Fluxo Principal do Usuário

```
[Login / Cadastro]
  ↓
[Dashboard – visão geral dos sistemas cadastrados]
  ↓
[Selecionar Sistema] ↔ [Cadastrar Novo Sistema]
  ↓
[Gerenciar Requisitos – definir cotas e tolerâncias]
  ↓
[Abrir Ciclo de Teste – informar versão e ambiente]
  ↓
[Registrar Medições por Requisito]
  ├── Desvio calculado automaticamente
  ├── Status: Aprovado / Alerta / Reprovado
  ↓
[Encerrar Ciclo]
  ↓
[Laudo de Conformidade gerado automaticamente]
  ├── Índice de Conformidade: XX%
  ├── Classificação: Conforme / Condicional / Não-Conforme
  ↓
[Exportar PDF] ↔ [Comparar com versão anterior]
```

3.2 Fluxos Alternativos

Requisito sem medição no ciclo: O sistema exibe alerta indicando quais requisitos ainda não possuem medição registrada. O encerramento do ciclo é bloqueado até que todos os requisitos ativos sejam medidos.

Valor medido fora de tolerância crítica: O sistema exibe destaque visual em vermelho para a medição reprovada. Se o requisito for de criticidade Crítica, a classificação final do laudo é forçada para Não-Conforme independentemente do índice geral.

Tentativa de edição de ciclo encerrado: O sistema bloqueia qualquer tentativa de inserção ou edição de medições em ciclos com status "encerrado" e exibe mensagem explicativa.

Tentativa de acesso sem autenticação: Qualquer rota da API retorna HTTP 401. O frontend redireciona automaticamente para a tela de login.

4. Mockups e Experiência do Usuário (UX)

Esta seção apresenta **a visualização inicial do produto antes da implementação**.

4.1 Fluxo de Navegação

```
/login
  ↓
/dashboard      → visão geral: sistemas, ciclos abertos, laudos emitidos
  ↓
/sistemas       → lista de sistemas cadastrados
  ↓
/sistemas/:id   → detalhe do sistema: requisitos + ciclos
  ↓
/ciclos/:id     → ciclo ativo: tabela de medições por requisito
  ↓
/laudos/:id     → laudo de conformidade gerado
```

4.2 Wireframes ou Mockups das Telas

Tela 1 — Dashboard

- **Descrição:** Visão geral dos sistemas cadastrados pelo usuário
- **Elementos:** Cards com nome do sistema, status do último ciclo, índice de conformidade mais recente
- **Ações principais:** "+ Novo Sistema", acessar detalhe de um sistema existente

Tela 2 — Detalhe do Sistema

- **Descrição:** Lista de requisitos (cotas) e ciclos de teste do sistema selecionado
- **Elementos:** Tabela de requisitos com valor nominal e tolerâncias; lista de ciclos com status e data
- **Ações principais:** "+ Novo Requisito", "+ Novo Ciclo", acessar ciclo existente

Tela 3 — Ciclo de Teste (Registro de Medições)

- **Descrição:** Interface principal de coleta de dados de inspeção
- **Elementos:** Tabela com colunas: Requisito | Nominal | Tolerância Inf | Tolerância Sup | Valor Medido | Desvio | Status
- **Ações principais:** Inserir valor medido por linha; "Encerrar Ciclo e Gerar Laudo"

Tela 4 — Laudo de Conformidade

- **Descrição:** Documento formal de conformidade gerado ao encerrar o ciclo
- **Elementos:** Cabeçalho (sistema, versão, data, ambiente); tabela de medições com código de cores por status; Índice de Conformidade Geral com barra visual; classificação final em destaque
- **Ações principais:** "Exportar PDF", "Comparar com versão anterior"

4.3 Fluxo de Interação do Usuário

Fluxo completo de inspeção de um sistema:

1. Usuário acessa o sistema em <https://softmeter.app> e realiza login
2. No Dashboard, clica em "+ Novo Sistema" e cadastra o sistema a ser avaliado
3. No detalhe do sistema, define os requisitos: para cada um, informa descrição, valor nominal, tolerância superior, tolerância inferior e unidade de medida
4. Clica em "+ Novo Ciclo", informa a versão do software e descreve o ambiente de teste
5. Na tela do ciclo, insere o valor medido para cada requisito — o sistema calcula o desvio e exibe o status automaticamente
6. Ao finalizar, clica em "Encerrar Ciclo e Gerar Laudo"
7. O laudo é exibido com o índice de conformidade e a classificação final
8. O usuário exporta o laudo em PDF para entrega ao cliente

4.4 Feedback Inicial de Usuários

Durante a apresentação da proposta a profissionais de QA e gestores de TI, foram coletados os seguintes feedbacks:

- **"O conceito de tolerância faz total sentido — é exatamente o que falta nas nossas entregas."** — Analista de QA
- **"Ter um laudo com número é muito mais fácil de apresentar para o cliente do que um relatório subjetivo."** — Gestor de TI
- **Sugestão de melhoria:** incluir comparação visual entre laudos de versões diferentes do mesmo sistema — funcionalidade já prevista no escopo (RF12)

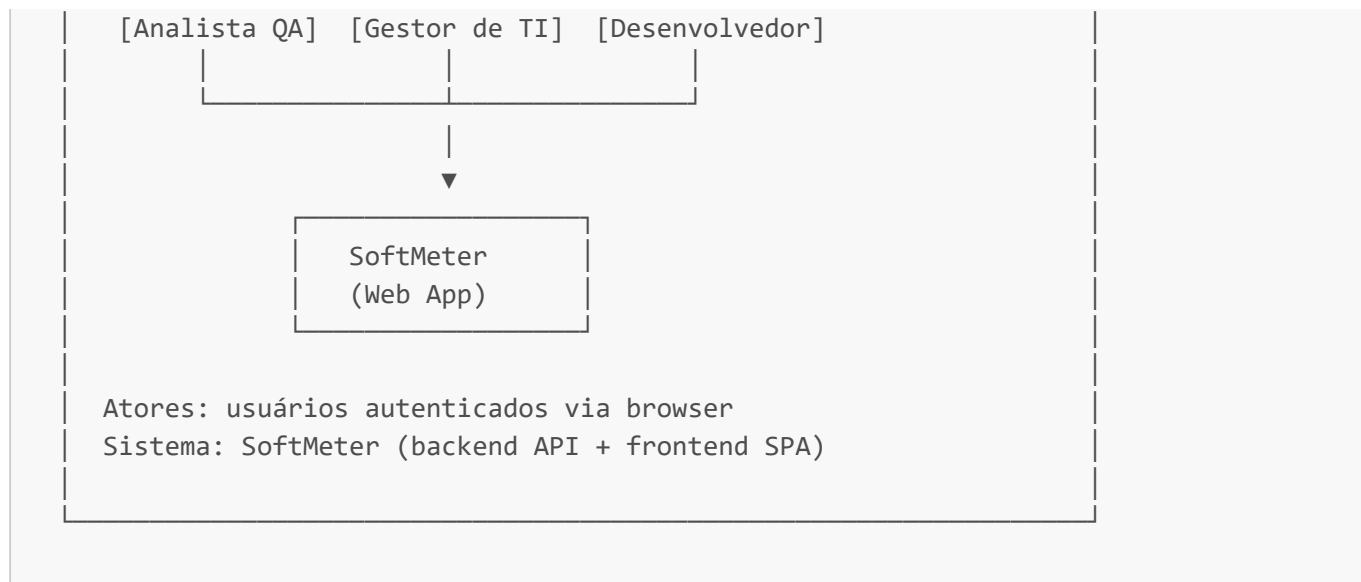
5. Arquitetura do Sistema

Esta seção demonstra **como o sistema será construído**.

5.1 Diagrama C4

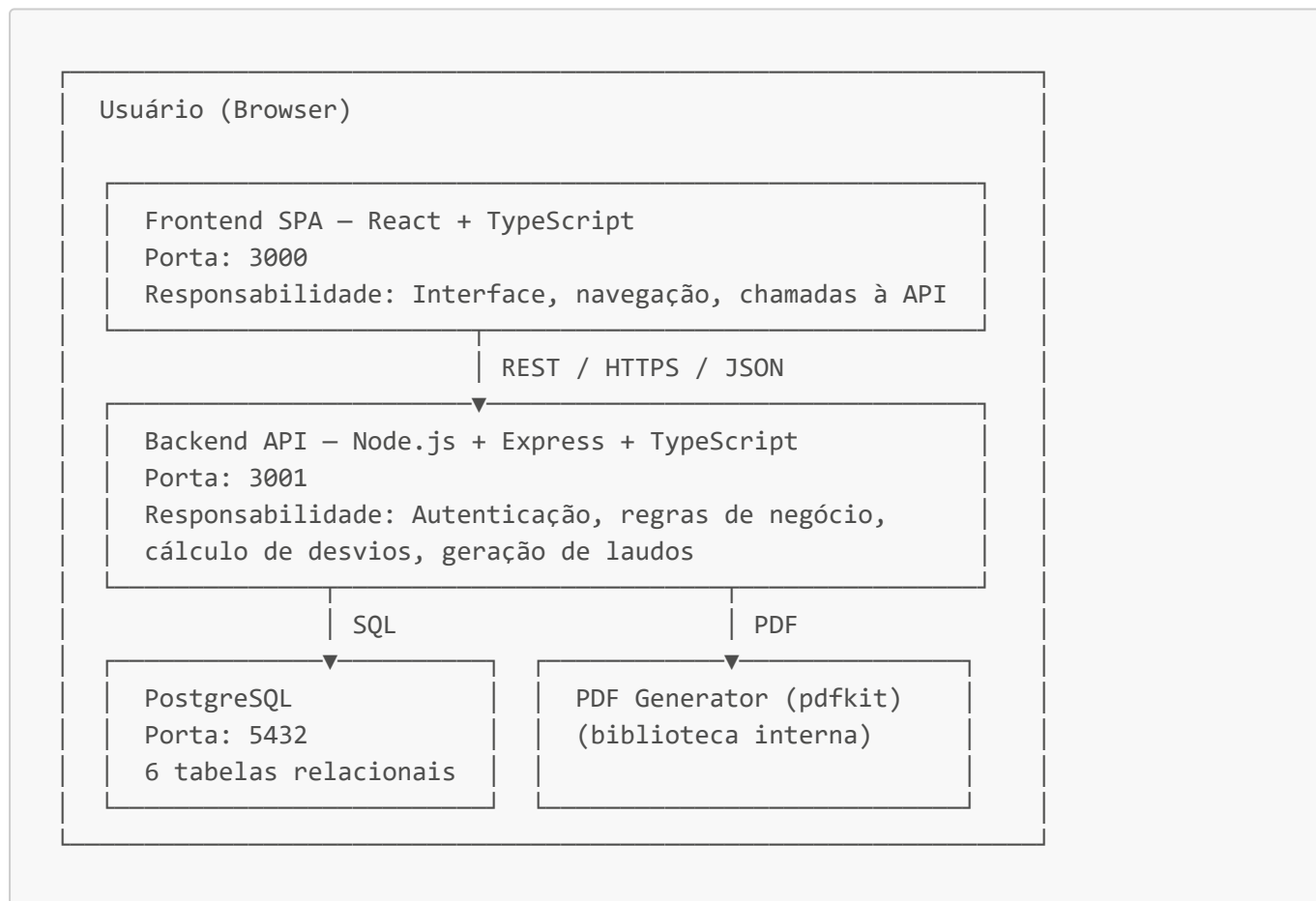
Nível 1 — Diagrama de Contexto



**Atores:**

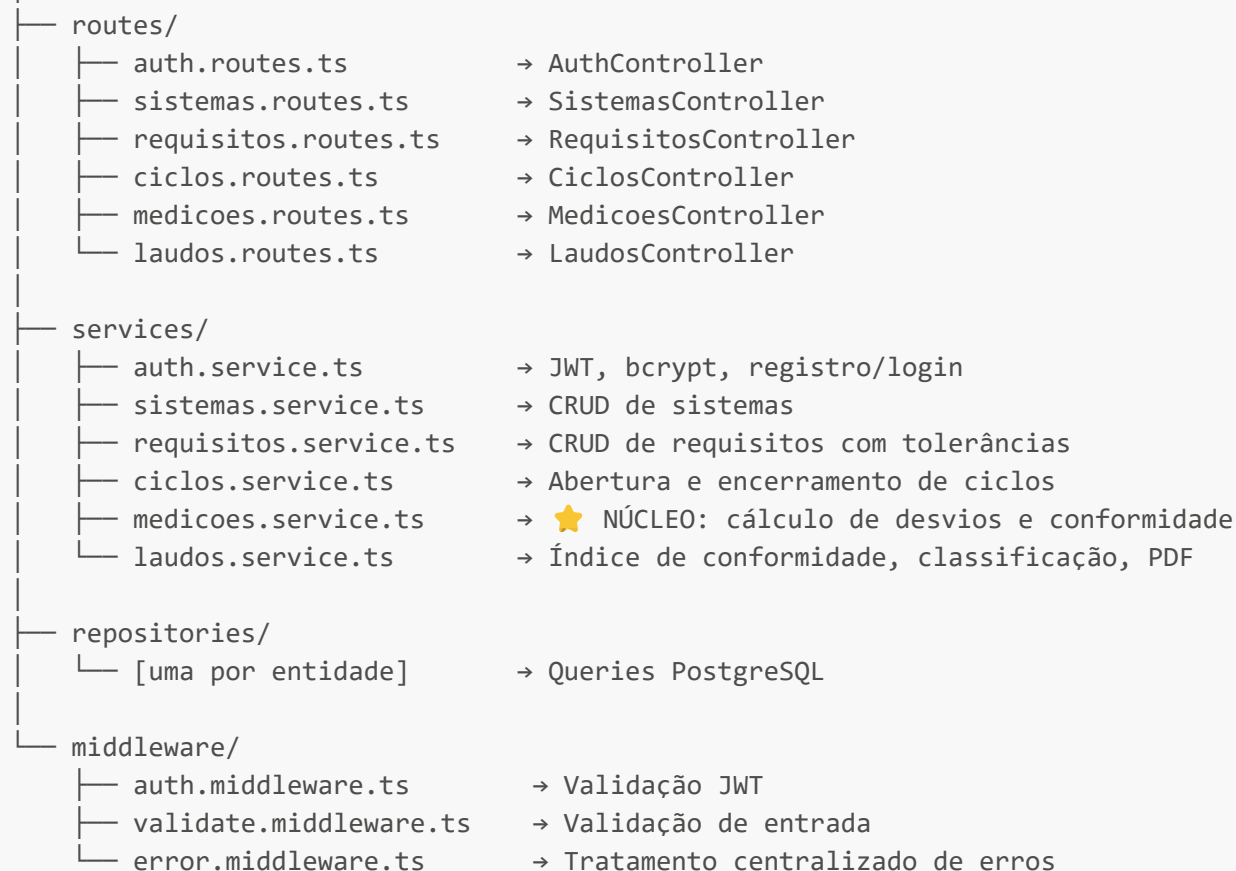
- **Analista de QA:** registra medições e gera laudos de conformidade
- **Gestor de TI:** consulta laudos e índices de conformidade para entrega ao cliente
- **Desenvolvedor:** verifica conformidade de requisitos antes do deploy

Sistemas externos: nenhum na versão inicial (dados inseridos manualmente pelo usuário)

Nível 2 — Diagrama de Containers

Nível 3 — Diagrama de Componentes (Backend API)

Backend API (Node.js + Express)



5.2 Modelo de Dados

Esquema relacional — 6 tabelas:

tb_usuarios (id, nome, email, senha_hash, created_at, updated_at)

tb_sistemas (id, id_usuario FK, nome, descricao, responsavel, status, created_at, updated_at)

tb_requisitos (id, id_sistema FK, codigo, descricao, valor_nominal, tolerancia_sup, tolerancia_inf, unidade, criticidade, ativo, created_at, updated_at)

tb_ciclos_teste (id, id_sistema FK, versao_software, ambiente, status, data_inicio, data_fim, created_at, updated_at)

tb_medicoes (id, id_ciclo FK, id_requisito FK, valor_medido, desvio_absoluto, desvio_relativo, status_conformidade, observacao, created_at)
 UNIQUE(id_ciclo, id_requisito)

```
tb_laudos (id, id_ciclo FK UNIQUE, total_requisitos, total_aprovados,  
           indice_conformidade, classificacao_final, pdf_url,  
           observacoes_gerais, created_at)
```

Relacionamentos:

- `tb_usuarios` (1) → (N) `tb_sistemas`
- `tb_sistemas` (1) → (N) `tb_requisitos`
- `tb_sistemas` (1) → (N) `tb_ciclos_teste`
- `tb_ciclos_teste` (1) → (N) `tb_medicoes`
- `tb_requisitos` (1) → (N) `tb_medicoes`
- `tb_ciclos_teste` (1) → (1) `tb_laudos`

5.3 Principais Componentes

Componente	Responsabilidade
Auth Service	Registro, login, geração e validação de JWT
Sistemas Service	CRUD de sistemas de software avaliados
Requisitos Service	CRUD de requisitos com valores nominais e tolerâncias
Ciclos Service	Abertura, gestão e encerramento de ciclos de teste
Medições Service	Registro de valores medidos e cálculo automático de desvios e conformidade
Laudos Service	Geração do índice de conformidade, classificação final e PDF do laudo

5.4 Stack Tecnológica

Node.js 20 LTS

Escolhido pela capacidade de lidar com alto volume de requisições I/O, ecossistema rico em bibliotecas e suporte nativo a TypeScript. Recomendado nas diretrizes do Portfólio da Católica SC.

TypeScript 5.x

Tipagem estática reduz bugs em tempo de desenvolvimento, melhora a legibilidade do código e garante contratos claros entre as camadas da aplicação — especialmente importante para os tipos metrológicos (`StatusConformidade`, `Criticidade`, `ClassificacaoFinal`).

Express.js 4.x

Framework minimalista e flexível para APIs RESTful. Escolhido pela maturidade, documentação abrangente e facilidade de configuração de middlewares de autenticação, validação e tratamento de erros.

PostgreSQL 16

Banco de dados relacional robusto, explicitamente preferido nas diretrizes do Portfólio. Suporte a transações ACID, índices eficientes e queries complexas necessárias para os cálculos de conformidade metrológica.

React 18 + TypeScript

Biblioteca de interface padrão de mercado, componentização eficiente, reatividade de estado e suporte nativo

a TypeScript. Permite construção incremental das telas da plataforma.

Docker + Docker Compose

Garante ambiente de desenvolvimento consistente e portátil entre máquinas. Listado como preferido nas diretrizes do Portfólio. Permite subir todo o ambiente (backend + frontend + banco) com um único comando.

Jest 29

Framework de testes padrão para o ecossistema Node.js/React. Utilizado para TDD (Test-Driven Development) com 17 testes unitários implementados no núcleo metrológico da aplicação.

GitHub Actions

CI/CD integrado nativamente ao repositório. Pipeline configurado para executar testes automaticamente a cada push nas branches main e develop.

6. Segurança e Privacidade

6.1 Proteções Implementadas

Autenticação e Autorização:

- JWT (JSON Web Tokens) com expiração configurável
- Tokens armazenados em memória no frontend (sem localStorage para evitar XSS)
- Middleware de autenticação aplicado a todas as rotas protegidas
- Senhas com hash bcrypt (salt rounds ≥ 12)

Proteção contra ataques OWASP Top 10:

- Injeção SQL: uso de queries parametrizadas (sem concatenação de strings)
- XSS: validação e sanitização de entradas no backend
- CSRF: tokens JWT stateless eliminam a superfície de ataque CSRF
- Rate limiting: a implementar nos próximos sprints
- HTTPS: obrigatório em produção via certificado SSL

Configuração segura:

- CORS configurado para aceitar apenas origens autorizadas
- Variáveis sensíveis gerenciadas via variáveis de ambiente (nunca no código-fonte)
- Segredos do CI/CD gerenciados via GitHub Secrets

6.2 Privacidade e LGPD

Dados coletados:

- Nome e e-mail do usuário (cadastro)
- Dados de sistemas e requisitos inseridos voluntariamente pelo usuário
- Logs de acesso para fins de monitoramento e segurança

Armazenamento:

- Banco de dados PostgreSQL com acesso restrito por credenciais
- Senhas nunca armazenadas em texto claro (bcrypt)
- Dados de sistemas pertencem exclusivamente ao usuário que os cadastrou

Direitos do titular (Art. 18, LGPD):

- O usuário pode solicitar exclusão da conta via interface, com remoção de todos os dados associados
- Não há coleta de dados sensíveis (saúde, financeiros, biométricos)
- O sistema processa dados técnicos de software, não dados pessoais de terceiros

7. Planejamento do Projeto

Marco	Descrição	Prazo
M1 — RFC aprovada	Documento de planejamento completo e revisado pelo comitê	Semana 2
M2 — Setup + Estrutura	Repositório configurado, Docker funcional, CI/CD básico, banco criado	Semana 3 ✓
M3 — Backend MVP	CRUD de sistemas, requisitos e ciclos com autenticação JWT	Semana 6
M4 — Medições + Desvios	Registro de medições com cálculo automático de desvios e conformidade	Semana 8
M5 — Laudos	Geração do Laudo de Conformidade com exportação PDF	Semana 10
M6 — Frontend	Interface React funcional e integrada ao backend	Semana 13
M7 — Testes + Qualidade	Cobertura ≥ 75%, pipeline completo, SonarCloud configurado	Semana 15
M8 — Deploy + Poster	Deploy em produção, poster técnico, Demo Day	Semana 18

Status atual: M2 concluído — ambiente Docker com backend (Node.js/TypeScript), frontend (React) e banco PostgreSQL (6 tabelas) funcionando localmente. Pipeline CI/CD configurado. 17 testes unitários implementados e passando.

8. Referências

- BASILI, Victor R.; CALDIERA, Gianluigi; ROMBACH, H. Dieter. **Goal Question Metric Paradigm**. Encyclopedia of Software Engineering, New York: Wiley, v. 1, p. 528–532, 1994.
- BUREAU INTERNATIONAL DES POIDS ET MESURES. **Vocabulário Internacional de Metrologia (VIM)**, 3ª ed. Duque de Caxias: INMETRO, 2012.
- ISO/IEC 25010:2023. **Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model**. Geneva: ISO, 2023.

- PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de Software: uma abordagem profissional**. 8. ed. Porto Alegre: AMGH, 2016.
- SOMMERVILLE, Ian. **Engenharia de Software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011.
- WINCK, Diogo Vinícius. **Mais Que Código: Um Manifesto para Projetos de Conclusão de Curso**. Medium, fev. 2026. Disponível em: <https://medium.com/@diogo.winck>. Acesso em: 12 abr. 2026.
- OWASP Foundation. **OWASP Top Ten**. Disponível em: <https://owasp.org/www-project-top-ten/>. Acesso em: 12 abr. 2026.

9. Apêndices

Apêndice A — Paralelo Metrologia Industrial × SoftMeter

Metrologia Industrial	SoftMeter (Equivalente)
Desenho técnico / GD&T	Documento de Requisitos
Tolerância dimensional	Tolerância por requisito (sup/inf)
Medição com Braço FARO	Coleta de valores em ciclo de testes
Software PolyWorks	SoftMeter — análise e laudo
Laudo de Inspeção	Laudo de Conformidade de Software
Peça aprovada / refugada	Software Conforme / Não-Conforme
Incerteza de medição	Variabilidade de performance entre ambientes
Calibração do instrumento	Padronização do ambiente de testes

Apêndice B — Exemplo de Laudo de Conformidade

Característica	Nominal	Tol. Inf	Tol. Sup	Medido	Desvio	Status
Tempo de Login	2,00 s	1,50 s	2,50 s	2,35 s	+0,35 s	✓ Aprovado
Precisão de Cálculo (R\$)	R\$ 100,00	R\$ 100,00	R\$ 100,00	R\$ 100,01	+R\$ 0,01	✗ Reprovado
Taxa de Erro de Input	0%	0%	2%	1,5%	-0,5%	✓ Aprovado
Disponibilidade do Sistema	99,9%	99,0%	100%	98,7%	-1,2%	✗ Reprovado

Índice de Conformidade: 50% → Classificação: ✗ Não-Conforme
(Nota: exemplo didático com dados fictícios para ilustração do formato do laudo)

Apêndice C — Repositório e Links

- **Repositório GitHub:** <https://github.com/DC811cunha/metrologia-software>
- **Ambiente local — Frontend:** <http://localhost:3000>
- **Ambiente local — API (health check):** <http://localhost:3001/health>

10. Parecer do Comitê de Avaliação

(A ser preenchido pelos professores)

PAULO ROGERIO PIRES MANSEIRA: _____

Status: ☐ Aprovado ☐ Ajustar

Observações:

LUIZ CARLOS CAMARGO: _____

Status: ☐ Aprovado ☐ Ajustar

Observações: